

Course Title	Code	Semester	Hours	Credits	ECTS
Advanced Database Systems	BLG 564E	Spring	3	3	7.5
Prerequisites	Undergraduate Database Systems course recommended				
Language of Instruction	English				
Course Type (Required /elective)	Elective				
Instructors /e-mail / Office Hour	Ali Çakmak / ali.cakmak@itu.edu.tr / Office hours: Tue: 14-16				
Course Content	- Advanced working principles of traditional relational database management systems (query optimization, transaction processing, concurrency control). - Design principles of modern data architectures including Data Lakehouses (e.g., Iceberg, Delta Lake) and Cloud-Native Databases (separation of compute and storage). - In-depth analysis of NoSQL and Vector Databases for high-dimensional data and AI applications (e.g., embeddings, nearest neighbor search). - Introduction to AI-driven database components (learned indexes, learned optimizers) and real-time Stream Processing architectures.				
Learning Outcomes	<i>Query Optimization:</i> Given a query and database statistics, compare and critically evaluate alternative query execution plans, and explain how AI/ML-based optimizers differ from traditional cost-based models. <i>Transaction Management:</i> Explain how ACID properties are implemented in both single-node and distributed database systems (including consensus algorithms like Paxos/Raft and modern clock synchronization techniques). <i>Big Data Architectures:</i> Design a scalable data processing workflow using modern Stream Processing engines (e.g., Spark Structured Streaming, Flink). <i>Vector & NoSQL Systems:</i> Evaluate the trade-offs between different NoSQL models (Key-Value, Document, Column-family) and design a schema for a Vector Database to support similarity search and RAG (Retrieval-Augmented Generation) applications. <i>Modern Storage Engines:</i> Differentiate between Row-oriented vs. Column-oriented storage formats (e.g., Parquet) and understand the architectural shift towards Data Lakehouses and storage-compute separation				
Assessment Criteria	Assessment Component			Weight	
	Paper review presentations			10%	
	Mini Exams (Quizzes) (5 of them - 10% each)			50%	
	Term Project			40%	

WEEKLY SCHEDULE	
Weeks	Topics
Week 1	Review of database fundamentals

Week 2	Query Execution Algorithms (Selection, Sorting, Joins)
Week 3	Query Optimization (cost models, database statistics, cost-based query optimization)
Week 4	Query Optimization (materialization, pipelining, query transformation)
Week 5	Transaction Processing (ACID properties, Serializability. Concurrency control)
Week 6	Transaction Processing (Two-phase locking. Deadlock prevention, detection and recovery)
Week 7	Indexing + Learned Indexes (Traditional vs. ML based indexing)
Week 8	Distributed Databases (Horizontal/vertical partitioning. Commit protocol. Distributed concurrency control. Semi-join)
Week 9	Parallelism & Columnar Analytics (Modern OLAP Architecture, SIMD, Vectorization)
Week 10	Stream Processing & Real-time Data (Apache Flink, Spark Streaming, Event-Time processing)
Week 11	Data Lakehouse Architectures (Storage/Compute Separation, Apache Iceberg, Delta Lake)
Week 12	NoSQL & NewSQL (CAP Theorem, Consistency Models, Spanner/CockroachDB - Clock Synchronization)
Week 13	Vector Databases & Neural Search (HNSW, RAG Pipelines, Embeddings management)
Week 14	Project Presentations

DELIVERABLES, EXAMS, PRESENTATIONS	
Weeks	Deliverable
Week 3	Project Proposal
Week 4	Quiz 1
Week 6	Quiz 2
Week 7	- Paper Review Report - Paper Review Presentation
Week 9	Quiz 3
Week 10	Progress Report
Week 11	Quiz 4
Week 13	Quiz 5
Week 14	- Term Paper - Project Presentation

REFERENCE READINGS
- Ramez Elmasri and Shamkant B Navathe. Fundamentals of Database Systems. Hoboken, New Jersey: Pearson, 7th Ed., 2017. - Kleppmann, M. (2025). Designing data-intensive applications: The big ideas behind reliable, scalable, and maintainable systems. O'Reilly Media, Inc. - Silberschatz, Abraham, Henry F. Korth, and Sudarshan, S. Database System Concepts., 7th ed., New York: McGraw-Hill, 2019. - Bailis, Peter, Hellerstein, Joseph M., and Stonebraker, Michael. Readings in Database Systems, 5th Edition, ebook available at http://www.redbook.io/, 2015.

- Perkins, L., Redmond, E., & Wilson, J. (2018). Seven databases in seven weeks: a guide to modern databases and the NoSQL movement. Pragmatic Bookshelf.
- Kraska, T., Beutel, A., Chi, E. H., Dean, J., & Polyzotis, N. (2018). "The Case for Learned Index Structures." *Proceedings of the 2018 ACM International Conference on Management of Data (SIGMOD '18)*. [Started the "Learned Indexing" field, arguing that B-Trees can be replaced by neural networks]
- Marcus, R., et al. (2019). "Neo: A Learned Query Optimizer." *Proceedings of the VLDB Endowment*, 12(11). [Predicting better execution plans and replacing traditional cost models]
- Kipf, Andreas, et al. "Learned Cardinalities: Estimating Correlated Joins with Deep Learning." *Proceedings of CIDR 2019*.
- Malkov, Y. A., & Yashunin, D. A. (2020). "Efficient and Robust Approximate Nearest Neighbor Search using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence*. [HNSW, the underlying indexing algorithm for almost all modern vector databases (Milvus, Pinecone, Weaviate)].
- Johnson, J., Douze, M., & Jégou, H. (2021). "Billion-scale similarity search with GPUs." *IEEE Transactions on Big Data*. [Faiss, a library standard for efficient similarity search, crucial for RAG (Retrieval-Augmented Generation) architectures]
- Armbrust, M., et al. (2021). "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics." *Proceedings of CIDR 2021*. [Introducing the Lakehouse architecture (Delta Lake/Iceberg), moving beyond the Hadoop/MapReduce paradigms].
- Verbitski, A., et al. (2017). "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases." *Proceedings of SIGMOD '17*. [Essential for understanding "Separation of Compute and Storage", the standard design pattern for modern cloud databases]
- Lamb, A., et al. (2012). "The Vertica Analytic Database: C-Store 7 Years Later." *Proceedings of the VLDB Endowment*, 5(12). [Understanding Columnar Stores (C-Store), which underpins modern OLAP systems (ClickHouse, Snowflake, BigQuery)].
- Akidau, T., et al. (2015). "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing." *Proceedings of the VLDB Endowment*, 8(12). [Modern streaming concepts like "Watermarks" and "Windowing," essential for IoT Data Streams]
- Carbone, P., et al. (2015). "Apache Flink: Stream and Batch Processing in a Single Engine." *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering*. [Moving from "Batch-only" (MapReduce) to "Stream-first" processing].

TERM PROJECT

Projects should be done individually. The goal is to produce a publishable paper draft at the end of the semester. You may choose to work on one of the project types: (i) survey/review paper, (ii) benchmarking paper, and (iii) research paper.

1. Survey paper: Choose a data management/databases topic and write academic review/survey paper that explains the problem, common approaches in the literature, how they compare in terms of different dimensions, their advantages and disadvantages, and open research problems in the area. The criteria to select a particular field/problem:

- The problem field should be a hot topic (One measure to check if a subject is hot topic could be the following: Check if there have been at least 20 papers published on the topic within the last 5 years).
- There should not be a survey within the last 5 years in the same area. Or, if there is a survey paper on the same topic that has been published within the last 5 years, you should be able to clearly differentiate your paper from the existing one by filling important gaps in it.

Grading:

- ✓ Abstract & Intro: 15%
- ✓ Survey Content: 70%
- ✓ Presentation: 15%

Project Ideas:

- Databases to handle genomics data
- Cloud-Native Databases
 - Serverless Databases: Database systems that automatically scale based on demand, reducing operational overhead.
 - Multi-Cloud and Hybrid Cloud Data Management: Strategies for managing data across multiple cloud providers or hybrid environments.
 - Cost Optimization: Techniques for minimizing storage and compute costs in cloud databases.
- Data Integration and Interoperability
 - Architectures for Integrating Health Datasets
 - Data Lakes and Data Mesh: Architectures for integrating and managing data across diverse sources and formats.
 - Schema Mapping and Data Fusion: Techniques for combining data from heterogeneous sources with different schemas.
- Data Privacy and Security
 - Differential Privacy: Techniques for analyzing data while preserving individual privacy.
 - Secure Multi-Party Computation (SMPC): Enabling collaborative data analysis without sharing raw data.
 - Encrypted Databases: Querying encrypted data without decryption (e.g., homomorphic encryption).
 - GDPR and Compliance: Ensuring databases comply with data protection regulations.
- Machine Learning and AI in Data Management
 - AutoML for Databases: Automating database tuning, indexing, and query optimization using machine learning.
 - Data Cleaning and Preparation: Using AI to detect and correct errors in datasets.
 - AI-Driven Query Optimization: Leveraging ML to improve query performance and resource allocation.
- Time-Series and Spatial Data Management
 - Time-Series Databases: Efficient storage and querying of time-series data for IoT, finance, and monitoring applications (e.g., InfluxDB, TimescaleDB).
 - Spatial and Spatiotemporal Data: Managing and analyzing geographic and movement data for applications like urban planning and logistics.
- Blockchain and Decentralized Data Management
 - Decentralized Databases: Systems for managing data in distributed and trustless environments.

- Blockchain for Data Provenance: Ensuring data integrity and traceability using blockchain technology.
- Smart Contracts and Databases: Integrating databases with blockchain-based smart contracts.

➤ Query Optimization and Processing

- Adaptive Query Processing: Techniques for dynamically optimizing queries based on runtime conditions.
- Approximate Query Processing: Providing fast, approximate results for large datasets.
- Querying Unstructured Data: Techniques for querying text, images, and other unstructured data types.

➤ Data Governance and Quality

- Data Lineage and Provenance: Tracking the origin and transformation of data.
- Data Quality Management: Detecting and resolving data quality issues (e.g., missing values, inconsistencies).
- Metadata Management: Efficiently managing metadata for large-scale datasets.

➤ Edge and IoT Data Management

- Edge Databases: Managing data at the edge of the network for low-latency applications.
- IoT Data Streams: Real-time processing and storage of data from IoT devices.
- Federated Learning: Training machine learning models on distributed IoT devices without centralizing data.

➤ Ethical and Responsible Data Management

- Bias and Fairness: Ensuring datasets and algorithms are free from bias.
- Explainable Data Management: Making database operations and decisions transparent and interpretable.
- Sustainability: Reducing the environmental impact of data storage and processing.

➤ Quantum Databases

- Quantum Query Processing: Exploring how quantum computing can revolutionize database operations.
- Quantum-Safe Encryption: Preparing databases for the post-quantum era.

➤ Human-in-the-Loop Data Management

- Interactive Data Exploration: Tools for enabling users to explore and analyze data interactively.
- Crowdsourced Data Management: Leveraging human input for tasks like data cleaning and labeling.

Benchmarking paper: Choose a data management/databases problem for which several algorithms/approaches are published. Benchmark different approaches on various public datasets from different dimensions such as performance, accuracy, etc. The criteria to select a particular field/problem:

- There are at least 5 different approaches proposed to solve the problem in the literature. These approaches should ideally either have public GitHub repos or should be easy to implement to finish the project by the end of the semester.

- There is no benchmarking paper published within the last 5 years on the same topic covering the intended methods. Or, if there is a benchmarking paper that has been published within the last 5 years, you can clearly differentiate your paper from the existing one by filling important gaps in it.
- There are at least 3 different public datasets that you can benchmark the approaches on.

Grading:

- ✓ Abstract & Intro: 15%
- ✓ Related Work: Evaluated in the second grading item
- ✓ Methods: 10%
- ✓ Results: 60%
- ✓ Presentation: 15%

Project Ideas:

- Benchmarking data management systems for storing and querying genomic data
- Adaptive Query Processing: Benchmark systems that dynamically optimize queries based on runtime conditions.
- Approximate Query Processing: Evaluate the trade-off between query accuracy and response time in approximate query processing systems.
- Querying Unstructured Data: Test the performance of querying text, images, and other unstructured data types.
- Benchmark systems like Apache Kafka, Apache Flink, and Apache Storm for handling real-time data streams.
- Benchmark NewSQL Databases like CockroachDB and TiDB for scalability and ACID compliance.
- Cloud-Native Database Benchmarking
 - Serverless Databases: Evaluate the performance and cost-effectiveness of serverless database offerings like AWS Aurora Serverless and Google Firestore.
 - Multi-Cloud Databases: Benchmark systems that operate across multiple cloud providers (e.g., YugabyteDB, Citus).
 - Auto-Scaling and Elasticity: Test how well cloud databases scale up and down based on workload demands.
- Time-Series Databases Benchmarking: Compare systems like InfluxDB, TimescaleDB, and Prometheus for storing and querying time-series data.
- Spatial Databases Benchmarking: Evaluate systems like PostGIS and Oracle Spatial for spatial query performance and scalability.
- AI-Driven Query Optimization: Benchmark the effectiveness of machine learning techniques in optimizing query performance.
- Data Cleaning and Preparation: Evaluate tools and frameworks for automating data cleaning and preprocessing tasks.
- Model Training on Databases: Measure the performance of in-database machine learning frameworks like Apache MADlib and Google BigQuery ML.
- Encrypted Databases: Evaluate the performance of systems that support encrypted query processing (e.g., homomorphic encryption).
- Differential Privacy: Benchmark the trade-off between data utility and privacy preservation in differentially private systems.
- Access Control and Auditing: Test the performance and scalability of access control mechanisms and auditing tools.
- Blockchain Performance: Measure the throughput and latency of blockchain-based databases for different consensus algorithms (e.g., PoW, PoS).
- Smart Contract Execution: Benchmark the performance of executing smart contracts on blockchain platforms like Ethereum and Hyperledger.
- Data Provenance and Integrity: Evaluate systems for ensuring data integrity and traceability using blockchain technology.

- Edge Databases: Test the performance of databases designed for edge computing environments.
- IoT Data Ingestion and Processing: Benchmark systems for handling high-velocity data streams from IoT devices.
- Federated Learning: Evaluate the performance and scalability of federated learning frameworks in distributed IoT environments.
- Data Lake Performance: Measure the performance of querying and analyzing data stored in data lakes.
- Schema Mapping and Data Fusion: Benchmark tools and techniques for integrating data from heterogeneous sources.
- ETL and ELT Pipelines: Evaluate the performance of ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) processes.
- Quantum Query Processing: Evaluate the performance of quantum algorithms for database operations.
- Quantum-Safe Encryption: Benchmark the performance of post-quantum encryption techniques in databases.

Research paper: Choose a database/data management problem. Develop a novel solution for this problem and evaluate it on different datasets in terms of performance, accuracy, scalability, etc. The criteria for selecting a particular problem:

- The problem is not studied in the literature yet, or the existing approaches could be improved with a better/different method.
- You will be able to experimentally compare your approach to at least two state-of-the-art approaches on the same dataset. For this item, you will need to check the following:
 - The state-of-the-art approaches have public GitHub repos that you can run on your datasets.
 - Or, it is easy to implement the state-of-the-art approaches from scratch.
 - Or, it is not required to re-run the state-of-art approaches, because you will be testing your approach exactly on the same dataset under the same settings that they use in their papers. Hence, you can directly compare against the results in their respective papers.

Grading:

- ✓ Abstract & Intro: 15%
- ✓ Related Work: Evaluated in the second grading item
- ✓ Methods: 30%
- ✓ Results: 40%
- ✓ Presentation: 15%

Project Ideas:

- The query optimizer in a database system is responsible for coming up with the most efficient execution plan for a given query. The goal of this project is to predict:
 - in the first phase, whether the query optimizer's cardinality estimates are off,
 - in the second phase, to what extent they are off (could be graded as "Slight", "So-so", "Significant"), and
 - in the third phase, whether there could be significantly better performing plan than the currently chosen plan.
 - in the fourth phase, whether dynamic sampling could be useful
- SQL LIKE queries take a textual pattern with wildcard characters and return the rows that match to the provided pattern. Estimating the selectivity of such queries is challenging. A promising direction may be adapting recent NLP models to solve this problem.

- An important step during query optimization is to come up with the best join order between different tables. Due to the large search space, the query optimizer spends significant amount of time to come up with the best join order. The goal this project is to mine the join order sequence patterns that are previously chosen for the past queries, and use them to reduce the join order optimization time for the future queries.
- Predicting when to stop the query optimization process in the middle without exploring the whole search space when the expected gain over the current best plan is “ignorable”.
- Recomputing the database stats in the catalog from the already executed query results.
- Predicting stats based on the trends of the past statistics.
- Finding the optimal time interval to gather stats for a database object based on stats changing percentage, data update characteristics, the effect of the stats on queries, etc.
- *Training Data Generation for the LIKE Query Selectivity Estimation:* LIKE query selectivity estimation is an important problem in relational databases. There are many works [1,2,3,4] in the literature that tried to solve the LIKE query selectivity estimation problem. The SQL LIKE is a logical operator that returns the tuples from a relation that match a specified pattern. For example, a SQL query "LIKE AIS" returns the all tuples in a relation that exactly match with string "AIS". LIKE operator uses two wildcards ("% and "_") in conjunction with LIKE predicates in order to have flexible patterns that may match a wide array of tuples in a relation. The percent sign represents zero, one or multiple characters and the underscore sign indicates a single character. "LIKE %AIS" query returns the all tuples in a relation that end with suffix "AIS" and "LIKE A_S" query matches tuples that have length three in a relation where the first character is "A" and last character is "S".

Recently, deep learning-based methods have become popular to estimate the selectivity of LIKE queries. However, generating training data (summary structure) for these methods is costly both in time and memory usage. Also, mostly the size of the train data is so large and it is not easy to train a model from a large train dataset.

The main goal of that project is to develop a method that generates proper train data for the problem of the LIKE query selectivity estimation. The project has three main steps. In the first step, it is expected to gain deep knowledge about the problem. In the second step, a new method should be proposed to generate the train data for the problem and at the last step, apply the current LIKE query selectivity estimation techniques to the generated train data and assess the effectiveness of the new method.

References

- 1: Aytimur, M., & Cakmak, A. (2018). Estimating the selectivity of LIKE queries using pattern-based histograms. *Turkish Journal of Electrical Engineering and Computer Sciences*, 26(6), 3319-3334.
- 2: Aytimur, M., & Cakmak, A. (2021). Using positional sequence patterns to estimate the selectivity of SQL LIKE queries. *Expert Systems with Applications*, 165, 113762.
3. Shetiya, S., Thirumuruganathan, S., Koudas, N., & Das, G. (2020). Astrid: accurate selectivity estimation for string predicates using deep learning. *Proceedings of the VLDB Endowment*, 14(4).
4. Kwon, S., Jung, W., & Shim, K. (2022). Cardinality Estimation of Approximate Substring Queries using Deep Learning. Technical Report. Seoul National University, Electrical and Computer Engineering Department.

- *A Novel Neural Embedding Approach for Frequent Sequence Pattern Representation:* The goal of this project is to develop and refine a state-of-the-art neural embedding technique that efficiently integrates both text-based syntax and pattern frequency information. This approach. The solution to be developed in this project could be directly employed for LIKE query selectivity estimation in databases.

Ref:

Nguyen, D., Luo, W., Nguyen, T. D., Venkatesh, S., & Phung, D. (2019). Sqn2vec: Learning sequence representation via sequential patterns with a gap constraint. In *Machine*

